

Build prompt — extending the Spurs Event Day Parking Alert MVP

Copy everything below the line into Claude Code (or another AI coding assistant) once you're ready to move past the manual-entry prototype.

You are extending an existing MVP called "Tottenham Hotspur Stadium Event Day Parking Alerts." It's a Streamlit app (app.py) backed by SQLite (db.py) with a rules engine (rules.py) that estimates when event-day parking restrictions start and end, and a cron-run script (notifier.py) that emails subscribers an advance notice and a same-day reminder to move their car.

Take this from a manual-entry personal tool to a real small product, in

phases. Confirm each phase works before moving to the next.

Phase 1: Automatic event data (partially done)

- `scraper.py` already pulls concert/NFL/other event candidates from tottenhamhotspurstadium.com's individual event pages, landing in a `scraped_events` review queue (admin confirms start time + type before anything reaches subscribers). Improve this:
 - The nav-link discovery approach is fragile — if the site restructures its "Concerts & Events" menu, links may be missed. Consider checking for a sitemap.xml or JSON endpoint the JS grid calls (inspect Network tab in browser devtools) as a more robust event-discovery source.
 - Add a lightweight scheduled run (e.g. once daily, respecting robots.txt) rather than only manual "Run scraper now" clicks.
- For football fixtures (Spurs home matches): use a football data API

(e.g. football-data.org, [API-Football](https://api-football.com)) — these are not on tottenhamhotspurstadium.com and need a separate integration.

- Whatever the source, keep the review-before-publish pattern already in place — never auto-publish scraped data straight to subscribers.

Phase 2: Accurate, zone-specific restriction times

- Haringey Council confirms exact bay suspension times per event about 7 days out (advertised on haringey.gov.uk). Investigate whether there's a structured page or feed for this rather than relying only on the generic defaults in [rules.py](#).
- Different CPZ zones (NPW, SA, BC, BGN, BGW, SL, 7S) may have different restriction windows for the same event — model this as configurable per-zone data rather than one global rule, if the evidence supports it.
- Keep the current conservative-default behavior as a fallback for any

event where zone-specific confirmed times aren't yet available.

Phase 3: Multi-channel notifications

- Add SMS via Twilio as an alternative/additional channel to email.
- Add a simple unsubscribe link in emails (currently unsubscribe is a manual form) — a one-click link avoids people typing their email wrong.

Phase 4: Postcode-based zone lookup

- Most people won't know their CPZ reference letters. Add a postcode lookup (or an address autocomplete) that maps to the correct zone, so registration doesn't require the user to already know "SA" vs "BGN".

Phase 5: Multi-tenant (other stadiums)

- Generalize the schema so this isn't Spurs-specific: a `venues` table, with `events` and zone-restriction-rules scoped per venue. This would let the same app serve residents near any stadium with matchday CPZs (there are several in London and other UK cities).

Constraints

- Never let this silently over-promise on timing accuracy — every subscriber-facing message about restriction times should communicate that these are estimates subject to official confirmation, and should point people to the council's own page for the final word.
- Keep `notifier.py` idempotent (already achieved via the `notifications_sent` table) — any redesign must preserve "never double-send the same notification."
- Add basic tests for `rules.py`'s time math before changing it, since a bug there directly causes people to get fined.

Start with Phase 1. Show me your plan for sourcing
football fixtures
before writing the integration code.